

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 Формирование технологического стека	5
2 Работа с базой знаний	6
2.1 Анализ структуры документа	6
2.2 Чанкирование	7
2.3 Векторизация	8
2.4 Сохранение в векторную БД.....	9
3 Формирование запроса к <i>LLM</i> и промпт-инжиниринг	10
3.1 Пайплайн извлечения	10
3.2 Реранкинг	12
3.3 Системный промпт.....	12
3.4 Генерация ответа.....	13
4 Разработка <i>API</i>	13
5 Разработка веб-интерфейса	14
ЗАКЛЮЧЕНИЕ	16
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	18
ПРИЛОЖЕНИЕ А Листинг программы	19

ВВЕДЕНИЕ

Данный подход, объединяющий алгоритмы семантического поиска и генеративные возможности нейросетей, демонстрирует наибольшую эффективность при работе со строго структурированными корпоративными и нормативно-правовыми документами, где требуется высокая точность извлечения фактов.

Целью данной работы является формирование и закрепление практических навыков проектирования комплексной *RAG*-системы на языке *Python*. Проект охватывает реализацию как серверной части для обработки запросов и взаимодействия с языковой моделью, так и клиентского веб-приложения.

Для достижения поставленной цели были определены следующие задачи.

- 1) Выбор и обоснование технологического стека на базе открытого (*open-source*) программного обеспечения, исключающего коммерческие затраты на развертывание инфраструктуры.

- 2) Подготовка пользовательской базы знаний на основе текста Конституции Российской Федерации: структурный анализ исходного документа, разбиение на текстовые фрагменты (чанкирование) с извлечением метаданных и последующая векторизация.

- 3) Построение программного конвейера (пайплайна), включающего поиск релевантных фрагментов, их переранжирование (перанжинг) и формирование структурированного системного промпта для минимизации фактологических ошибок (галлюцинаций) модели.

- 4) Проектирование и разработка *REST API* для интеграции логики *RAG*-системы.

5) Разработка веб-интерфейса для удобного пользовательского взаимодействия с созданной правовой базой знаний и вывода ответов с указанием источников.

1 Формирование технологического стека

В соответствии с требованиями задания, для реализации *RAG*-системы был сформирован комплекс инструментов на базе открытого программного обеспечения (*open-source*). Выбранные компоненты полноценно поддерживают работу с русским языком, не требуют оформления платных подписок и (за исключением векторного хранилища) предназначены для локального исполнения.

Архитектура системы включает следующие технологии.

- Модель векторизации: для генерации эмбеддингов задействована модель *paraphrase-multilingual-mpnet-base-v2* из пакета *sentence-transformers* [1]. Данное мультиязычное решение, базирующееся на архитектуре *BERT*, профилировано именно для задач семантического поиска. Модель трансформирует текст в векторные представления размерностью 768, демонстрирует высокую эффективность при обработке русскоязычных текстов и поддерживает аппаратное ускорение на *GPU*.

- Языковая модель (*LLM*): в качестве генеративного модуля применяется локальная нейросеть *qwen2.5:7b*, работающая через платформу *Ollama* [2]. Это открытая модель с 7 миллиардами параметров, отличающаяся качественной генерацией на русском языке. Локальный запуск обеспечивает конфиденциальность: обработка запросов происходит непосредственно на устройстве пользователя без отправки данных на внешние облачные серверы.

- Векторная база данных: Хранение вычисленных векторов и сопутствующих метаданных реализовано на базе сервиса *Qdrant Cloud* [3]. Использование облачного инстанса СУБД гарантирует повсеместный доступ к проиндексированной базе знаний с любого устройства, избавляя от необходимости проводить ресурсоемкую процедуру векторизации повторно.

- Модель переранжирования (Реранкер): для повышения точности извлечения релевантного контекста в пайплайн интегрирован *DiTy/cross-*

encoder-russian-msmarco [4]. Этот русскоязычный кросс-энкодер применяется на этапе постобработки: после первичного поиска он осуществляет переупорядочивание результатов, применяя алгоритмы более глубокого анализа семантической близости между пользовательским промптом и найденным документом.

- Серверная часть (*Backend*): Программный интерфейс (*API*) спроектирован с использованием *FastAPI* [5] — современного *Python*-фреймворка, поддерживающего автоматическую валидацию типов и самодокументирование. Взаимодействие с логикой приложения осуществляется через единственный маршрут (эндпоинт) */ask*, который принимает входящий вопрос и возвращает сгенерированный ответ со списком источников.

- Клиентская часть (*Frontend*): Веб-интерфейс реализован с помощью библиотеки *Streamlit* [6]. Инструмент позволил в сжатые сроки разработать интерактивное чат-приложение, поддерживающее сохранение истории сообщений и удобное отображение извлеченных статей Конституции в виде раскрывающихся блоков.

Подводя итог, сформированный технологический стек всецело отвечает изначальным условиям поставленной задачи: он является бесплатным, не имеет сторонних коммерческих зависимостей и оптимально подходит для работы с русскоязычной нормативно-правовой базой.

2 Работа с базой знаний

2.1 Анализ структуры документа

Базой знаний для *RAG*-системы выступает текст Конституции Российской Федерации в формате *.docx*. Документ обладает чёткой иерархической структурой, что делает его идеальным кандидатом для применения методов чанкирования и последующего семантического поиска.

Текст Конституции РФ состоит из 142 статей, каждая из которых начинается с заголовка в формате «Статья *N*», где *N* – арабская цифра или дробное число (например, 67.1). Внутри статьи может содержаться деление на части, но для целей данного проекта минимальной смысловой единицей принят полный текст одной статьи. Такой подход обеспечивает сохранение целостности правовой нормы.

Помимо основных статей, документ содержит статьи с дробной нумерацией (67.1, 75.1, 79.1, 92.1, 103.1), добавленные в результате конституционных поправок 2020 года, а также сноски с историческими комментариями.

2.2 Чанкирование

Для разбиения текста Конституции на отдельные статьи реализована функция *parse_articles()* в файле *load_and_chunk.py*. Функция использует регулярное выражение `r'Статья\s+(\d+(?:\.\d+)?)'` для корректного распознавания как целых (67), так и дробных (67.1) номеров статей.

Каждый фрагмент (чанк) сохраняется как словарь, содержащий текст статьи и метаданные: номер статьи и название главы. Для определения главы реализована функция *find_chapter()*, которая сопоставляет номер статьи с диапазонами статей каждой из 9 глав Конституции.

Для предотвращения появления дублирующихся чанков, возникающих из-за сносок и исторических комментариев, реализована дедупликация: при обнаружении повторного вхождения номера статьи сохраняется только первое. В результате чанкирования получено 142 уникальные статьи, что соответствует 137 основным статьям Конституции и 5 дробным статьям поправок 2020 года. Результирующие чанки сохраняются в файл *data/chunks.json* для последующего использования.

Фрагмент кода чанкирования показан на рисунке 1.

```

12 def parse_articles(text: str) -> list[dict]:
13     chunks = []
14
15     pattern = r'Статья\s+(\d+(?:\.\d+)?)'
16     matches = list(re.finditer(pattern, text))
17
18     for i, match in enumerate(matches):
19         article_id = match.group(1)
20         start = match.start()
21
22         if i + 1 < len(matches):
23             end = matches[i + 1].start()
24         else:
25             end = len(text)
26
27         article_text = text[start:end].strip()
28
29         if len(article_text) < 20:
30             continue
31
32         chunks.append({
33             "article_num": article_id,
34             "text": article_text,
35         })
36
37     seen = set()
38     unique_chunks = []
39     for chunk in chunks:
40         if chunk["article_num"] not in seen:
41             seen.add(chunk["article_num"])
42             unique_chunks.append(chunk)
43
44     return unique_chunks
45
46
47 def find_chapter(text: str, article_num: str) -> str:
48     chapter_pattern = r'ГЛАВА\s+(\d+)\s*\.s*\n(?:\n+)'
49     article_pattern = r'Статья\s+(\d+(?:\.\d+)?)'
50
51     chapters = list(re.finditer(chapter_pattern, text))
52
53     try:
54         current_art_val = float(article_num)
55     except ValueError:
56         return "Неизвестно"
57
58     for j, ch_match in enumerate(chapters):
59         ch_start = ch_match.start()
60         ch_end = chapters[j+1].start() if j+1 < len(chapters) else len(text)
61         chapter_text = text[ch_start:ch_end]
62
63         articles_in_chapter = re.findall(article_pattern, chapter_text)
64         if articles_in_chapter:
65             first_article = float(articles_in_chapter[0])
66             last_article = float(articles_in_chapter[-1])
67
68             if first_article <= current_art_val <= last_article:
69                 chapter_num = ch_match.group(1)
70                 chapter_name = ch_match.group(2).strip()
71                 return f"Глава {chapter_num}. {chapter_name}"
72
73     return "Неизвестно"

```

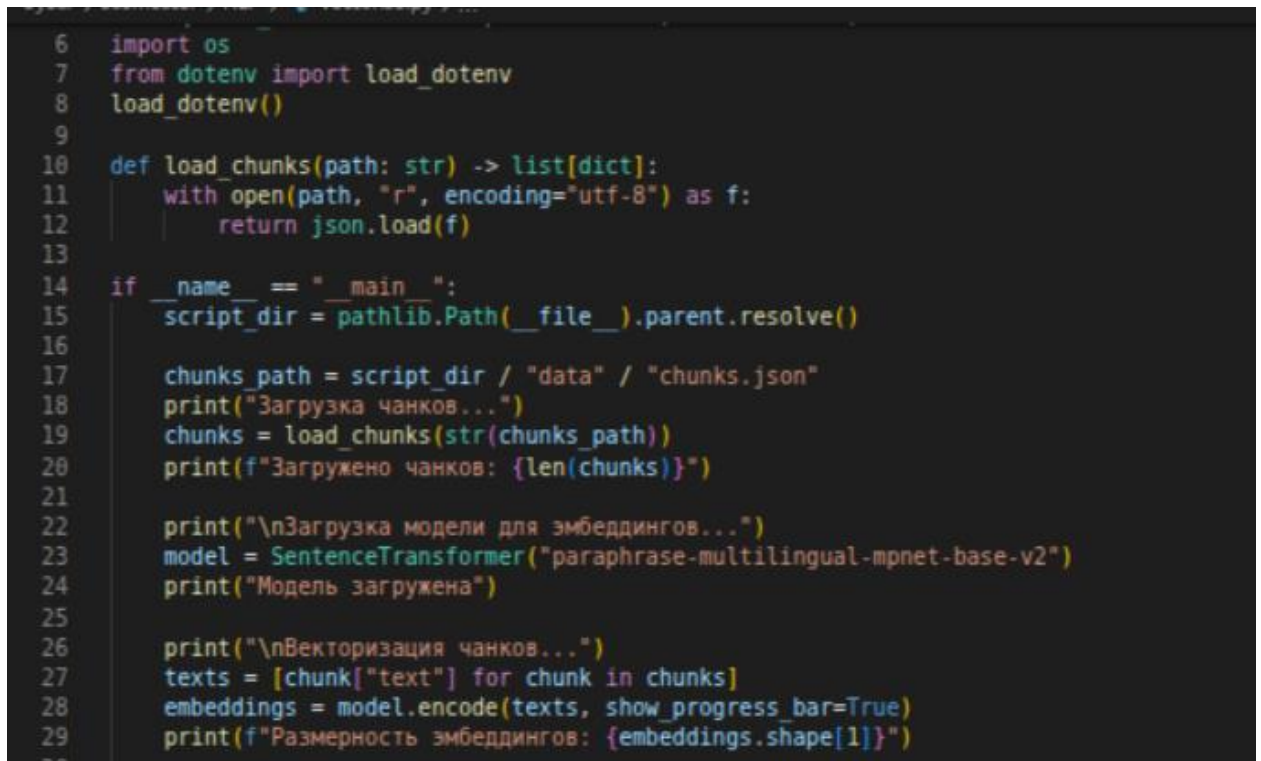
Рисунок 1 – Фрагмент кода чанкирования

2.3 Векторизация

После чанкирования каждая статья преобразуется в числовой вектор с помощью модели *paraphrase-multilingual-mpnet-base-v2* из библиотеки *sentence-transformers* [1]. Модель создаёт 768-мерные векторы, отражающие

смысловое содержание текста. Векторизация выполняется пакетно с использованием *GPU*, что позволяет обработать все 142 статьи за несколько секунд.

Фрагмент кода чанкирования показан на рисунке 2.

A screenshot of a code editor showing Python code for loading and processing JSON data. The code includes imports for 'os' and 'dotenv', a function 'load_chunks' to read JSON files, and a main block that loads the 'chunks.json' file, initializes a 'SentenceTransformer' model, and processes the text chunks into embeddings. The code is written in a dark-themed editor with syntax highlighting.

```
6 import os
7 from dotenv import load_dotenv
8 load_dotenv()
9
10 def load_chunks(path: str) -> list[dict]:
11     with open(path, "r", encoding="utf-8") as f:
12         return json.load(f)
13
14 if __name__ == "__main__":
15     script_dir = pathlib.Path(__file__).parent.resolve()
16
17     chunks_path = script_dir / "data" / "chunks.json"
18     print("Загрузка чанков...")
19     chunks = load_chunks(str(chunks_path))
20     print(f"Загружено чанков: {len(chunks)}")
21
22     print("\nЗагрузка модели для эмбедингов...")
23     model = SentenceTransformer("paraphrase-multilingual-mpnet-base-v2")
24     print("Модель загружена")
25
26     print("\nВекторизация чанков...")
27     texts = [chunk["text"] for chunk in chunks]
28     embeddings = model.encode(texts, show_progress_bar=True)
29     print(f"Размерность эмбедингов: {embeddings.shape[1]}")
```

Рисунок 2 – Фрагмент кода векторизации

2.4 Сохранение в векторную БД

Полученные векторы вместе с метаданными сохраняются в облачную базу данных *Qdrant Cloud* [3]. Для каждой статьи создаётся точка (*PointStruct*) с уникальным идентификатором, вектором и полезной нагрузкой (*payload*), содержащей номер статьи, название главы и полный текст. Коллекция создаётся с метрикой косинусного сходства. Всего было сохранено 142 документа.

Преимуществом облачного хранения является доступность базы данных с любого устройства по *API*-ключу – при переносе проекта на другой компьютер повторная векторизация не требуется.

Фрагмент кода загрузки в векторную БД представлен на рисунке 3.

```
31 client = QdrantClient(  
32     url=os.environ["QDRANT_URL"],  
33     api_key=os.environ["QDRANT_API_KEY"],  
34 )  
35  
36 collection_name = "constitution"  
37 existing = [c.name for c in client.get_collections().collections]  
38 if collection_name in existing:  
39     print(f"Коллекция '{collection_name}' уже существует, пересоздаём...")  
40     client.delete_collection(collection_name)  
41  
42 client.create_collection(  
43     collection_name=collection_name,  
44     vectors_config=VectorParams(  
45         size=embeddings.shape[1],  
46         distance=Distance.COSINE  
47     )  
48 )  
49 print(f"Коллекция '{collection_name}' создана")  
50  
51 points = [  
52     PointStruct(  
53         id=i,  
54         vector=embeddings[i].tolist(),  
55         payload={  
56             "article_num": chunk["article_num"],  
57             "chapter": chunk["chapter"],  
58             "text": chunk["text"]  
59         }  
60     )  
61     for i, chunk in enumerate(chunks)  
62 ]  
63  
64 print(f"\nЗагрузка {len(points)} векторов в Qdrant...")  
65 client.upsert(  
66     collection_name=collection_name,  
67     points=points  
68 )
```

Рисунок 3 – Фрагмент кода загрузки в векторную БД

3 Формирование запроса к *LLM* и промпт-инжиниринг

3.1 Пайплайн извлечения

После подготовки векторной базы знаний реализован пайплайн для обработки пользовательского запроса, включающий семантический поиск, реранжирование и генерацию ответа. На этапе разработки логика пайплайна была реализована и протестирована в файле *rag_pipeline.py*. После проверки корректности работы все функции были перенесены в файл *api.py*, где были доработаны и интегрированы непосредственно в серверную часть приложения.

На первом этапе выполняется семантический поиск в *Qdrant Cloud*. Система преобразует входной запрос пользователя в эмбединг с помощью той же модели *paraphrase-multilingual-mpnet-base-v2* и находит топ-15 наиболее семантически близких статей. Поиск выполняется по метрике косинусного сходства через метод *query_points()*.

Пример тестового вызова представлен на рисунке 4.

```
96 if __name__ == "__main__":
97     test_queries = [
98         "Какие права имеет человек на свободу слова?",
99         "Кто является верховным главнокомандующим?",
100         "Сколько лет должно быть президенту?"
101     ]
102
103     for query in test_queries:
104         print(f"\n{'='*60}")
105         print(f"Вопрос: {query}")
106         print(f'\n{'='*60}')
107         result = ask(query)
108         print(f"Ответ: \n{result['answer']}")
109         print(f"\nИсточники:")
110         for s in result["sources"]:
111             print(f" - Статья {s['article_num']} ({s['chapter']}) | rerank: {s['rerank_score']}")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

• ~/projects/OMGTU/3year/5semester/NLP_курс 3 python3 rag_pipeline.py
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF TOKEN to enable higher rate limits and faster
Loading weights: 100%
Loading weights: 100%

=====

Вопрос: Какие права имеет человек на свободу слова?

=====

Ответ:
Человек имеет право на свободу мысли и слова, согласно статье 29 Конституции Российской Федерации. Основные положения в этой с

1. Каждому гарантируется свобода мысли и слова (пункт 1).
2. Запрещается пропаганда социальной, расовой, национальной или религиозной ненависти и вражды; также запрещается пропаганда с
3. Никто не может быть принужден к выражению своих мнений и убеждений или отказу от них (пункт 3).

Таким образом, человек имеет право на свободное высказывание мыслей и информации, при этом ему запрещено распространять матери

Источники:
- Статья 29 (Глава 2. ПРАВА И СВОБОДЫ ЧЕЛОВЕКА И ГРАЖДАНИНА) | rerank: 0.2555
- Статья 21 (Глава 2. ПРАВА И СВОБОДЫ ЧЕЛОВЕКА И ГРАЖДАНИНА) | rerank: 0.2056
- Статья 2 (Глава 1. ОСНОВЫ КОНСТИТУЦИОННОГО СТРОЯ) | rerank: 0.159

=====

Вопрос: Кто является верховным главнокомандующим?

=====

Ответ:
Королество Российской Федерации Верховный Главнокомандующий Вооруженными Силами Российской Федерацией является Президент Росс
я гласит: "Президент Российской Федерации является Верховным Главнокомандующим Вооруженными Силами Российской Федерации."

Источники:
- Статья 87 (Глава 4. ПРЕЗИДЕНТ РОССИЙСКОЙ ФЕДЕРАЦИИ) | rerank: 0.2101
- Статья 113 (Глава 6. ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ) | rerank: 0.0552
- Статья 86 (Глава 4. ПРЕЗИДЕНТ РОССИЙСКОЙ ФЕДЕРАЦИИ) | rerank: 0.0022

=====

Вопрос: Сколько лет должно быть президенту?

=====

Ответ:
Президент Российской Федерации должен быть не моложе 35 лет. Этот минимальный возраст указан в статье 81 Конституции Российской

Номер статьи: 81

Источники:

Рисунок 4 – Пример тестового вызова

3.2 Реранкинг

Для повышения качества отбора использован русскоязычный реранкер *DiTy/cross-encoder-russian-msmarco* [4], основанный на архитектуре *cross-encoder*. В отличие от эмбединговой модели, которая кодирует запрос и документ независимо, *cross-encoder* обрабатывает пару «запрос – документ» совместно, что позволяет точнее оценить их релевантность.

Реранкер принимает запрос и 15 кандидатов от первичного поиска, оценивает каждую пару и возвращает список с оценками релевантности. Из переупорядоченного списка выбираются топ-5 наиболее релевантных статей, которые передаются на генерацию.

Важным практическим результатом применения реранкера стало решение проблемы нерелевантных запросов: для вопросов, не относящихся к Конституции РФ (например, «масса Солнца»), все статьи получают очень низкие оценки реранкера. Это используется для автоматической фильтрации нерелевантных запросов на уровне порогового значения (0.01).

3.3 Системный промпт

Системный промпт задаёт языковой модели чёткую ролевую модель: «Ты – юридический ассистент по Конституции РФ». В нём указаны обязательные правила генерации:

- 1) отвечать исключительно на русском языке;
- 2) формировать ответ только на основе предоставленных статей, не придумывая информацию;
- 3) если в контексте нет ответа – честно сообщать об этом;
- 4) указывать номера статей, на которые опирается ответ;
- 5) думать пошагово перед формулировкой финального ответа.

3.4 Генерация ответа

Этап генерации итогового ответа реализуется на базе локально развернутой языковой модели *qwen2.5:7b*, функционирующей в среде *Ollama* [2]. Программное взаимодействие с *LLM* осуществляется посредством интерфейса *ollama.Client*. В качестве входных данных нейросеть принимает составной промпт, включающий исходный запрос и извлеченный релевантный контекст, после чего формирует финальный текстовый результат.

В целях обеспечения стабильного качества вывода и обработки пограничных сценариев в конвейер внедрен этап программной постобработки ответа. Данная задача решается с помощью специально реализованной функции *clean_response()*. Её основное назначение — автоматическая фильтрация и удаление единичных вкраплений китайских иероглифов, которые периодически возникают в качестве артефактов генерации, что обусловлено изначальным происхождением семейства моделей *Qwen*.

4 Разработка API

Для обеспечения взаимодействия между веб-интерфейсом и RAG-системой разработан *REST API* на основе фреймворка *FastAPI* [5]. Серверная часть реализована в файле *api.py*.

При запуске сервера все модели (эмбединговая и реранкер) загружаются единожды с использованием механизма *lifespan* из *FastAPI*. Это исключает повторную загрузку весов при каждом запросе и обеспечивает быстрый отклик.

API предоставляет два эндпоинта. Корневой эндпоинт *GET /* возвращает статус сервера. Основной эндпоинт *POST /ask* принимает *JSON*-объект с полем *query* и возвращает объект с полями *answer* (сгенерированный текст) и *sources* (массив источников, каждый из которых содержит номер статьи, название

главы и оценку реранкера). Валидация входных и выходных данных выполнена с помощью *Pydantic*.

На уровне эндпоинта реализована двухуровневая фильтрация нерелевантных запросов. Первый уровень – проверка по ключевым фразам (вопросы о предыдущих сообщениях, системном промпте и т.д.). Второй уровень – проверка максимальной оценки реранкера: если ни одна статья не набрала оценку выше порогового значения 0.01, запрос признаётся нерелевантным и пользователь получает соответствующее сообщение без обращения к языковой модели.

Автоматически сгенерированная документация *Swagger UI* позволяет тестировать *API* непосредственно из браузера.

5 Разработка веб-интерфейса

Визуальная оболочка системы реализована на базе фреймворка *Streamlit* [6], основная логика которой сосредоточена в файле *app.py*. Интерфейс спроектирован в формате интерактивного чата, что обеспечивает привычный и интуитивно понятный способ взаимодействия пользователя с *RAG*-системой.

Интерфейс поддерживает историю сообщений, которая хранится в сессионном состоянии *Streamlit* (*st.session_state*). При каждом новом запросе приложение обращается к *API* по адресу *http://localhost:8000/ask*, после чего ответ отображается в чате. Под ответом размещается раскрывающийся блок «Источники», содержащий номера статей и названия соответствующих глав Конституции.

Во время ожидания ответа от сервера отображается индикатор загрузки с сообщением «Ищу ответ...». При недоступности *API*-сервера пользователь получает понятное сообщение об ошибке с рекомендацией проверить запуск сервера.

Веб-приложение запускается командой *streamlit run app.py* и доступно по адресу *http://localhost:8501*. Пример интерфейса приложения приведён на рисунке 5.

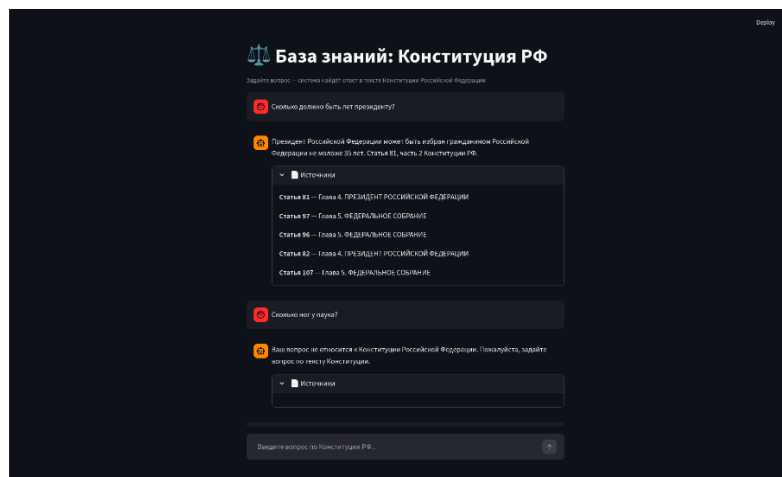


Рисунок 5 – Веб-интерфейс приложения на *Streamlit*

ЗАКЛЮЧЕНИЕ

В результате выполнения работы были изучены и практически реализованы фундаментальные принципы построения *RAG*-систем — одного из наиболее перспективных направлений в области разработки интеллектуальных вопросно-ответных сервисов. Был освоен современный стек технологий с открытым исходным кодом, охватывающий полный цикл разработки: от подготовки и векторизации неструктурированных текстовых данных до развертывания локальной языковой модели, проектирования *REST API* и создания пользовательского интерфейса.

В ходе проекта были успешно решены следующие задачи.

- 1) Сформирован технологический стек на основе *open-source* решений: *sentence-transformers*, *Qdrant Cloud*, *Ollama*, *FastAPI*, *Streamlit*.
- 2) Подготовлена база знаний: текст Конституции РФ разбит на 142 уникальные статьи с сохранением метаданных (номер статьи, название главы), векторизирован и сохранён в облачной векторной базе данных *Qdrant Cloud*.
- 3) Реализован пайплайн семантического поиска с применением русскоязычного реранкера *DiTy/cross-encoder-russian-msmarco*, что позволило значительно повысить точность отбора релевантных документов по сравнению с базовым векторным поиском.
- 4) Разработан системный промпт, обеспечивающий формирование ответов строго на основе предоставленного контекста, без галлюцинаций.
- 5) Реализован *REST API* с фильтрацией нерелевантных запросов и веб-интерфейс с историей диалога.

В процессе работы возникли следующие трудности. При разборе документа обнаружились статьи с дробной нумерацией (67.1, 75.1 и др.), что потребовало доработки регулярного выражения для чанкирования. При использовании английского реранкера *ms-marco-MiniLM-L-6-v2* наблюдалось неудовлетворительное качество ранжирования русскоязычных текстов: он не

мог найти статью 81 при вопросе о возрасте президента. Проблема была решена заменой на русскоязычный перанкер *DiTy/cross-encoder-russian-msmarco*. Также модель *qwen2.5* в некоторых случаях вставляла китайские иероглифы в ответ: для устранения этого реализована постобработка с удалением символов из диапазона *CJK Unicode*.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1 Hugging Face. paraphrase-multilingual-mpnet-base-v2. URL: <https://huggingface.co/sentence-transformers/paraphrase-multilingual-mpnet-base-v2> (дата обращения: 20.04.2026).
- 2 Ollama. qwen2.5. URL: <https://ollama.com/library/qwen2.5> (дата обращения: 20.04.2026).
- 3 Qdrant. Qdrant Documentation. URL: <https://qdrant.tech/documentation/> (дата обращения: 21.04.2026).
- 4 Hugging Face. DiTy/cross-encoder-russian-msmarco. URL: <https://huggingface.co/DiTy/cross-encoder-russian-msmarco> (дата обращения: 22.04.2026).
- 5 FastAPI. FastAPI Documentation. URL: <https://fastapi.tiangolo.com/> (дата обращения: 22.04.2026).
- 6 Streamlit. Streamlit Documentation. URL: <https://docs.streamlit.io/> (дата обращения: 23.04.2026).
- 7 Hugging Face. sentence-transformers. URL: <https://huggingface.co/sentence-transformers> (дата обращения: 20.04.2026).

ПРИЛОЖЕНИЕ А

(обязательное)

Листинг программы

Файл *load_and_chunk.py*

```
import re
from docx import Document
import json
import pathlib

def load_constitution(path: str) -> str:
    doc = Document(path)
    full_text = "\n".join([para.text for para in doc.paragraphs])
    return full_text

def parse_articles(text: str) -> list[dict]:
    chunks = []

    pattern = r'Статья\s+(\d+(?:\.\d+)?)'
    matches = list(re.finditer(pattern, text))

    for i, match in enumerate(matches):
        article_id = match.group(1)
        start = match.start()

        if i + 1 < len(matches):
            end = matches[i + 1].start()
        else:
            end = len(text)

        article_text = text[start:end].strip()

        if len(article_text) < 20:
            continue

        chunks.append({
            "article_num": article_id,
            "text": article_text,
        })

    seen = set()
    unique_chunks = []
    for chunk in chunks:
        if chunk["article_num"] not in seen:
            seen.add(chunk["article_num"])
```

```

        unique_chunks.append(chunk)

    return unique_chunks

def find_chapter(text: str, article_num: str) -> str:
    chapter_pattern = r'ГЛАВА\s+(\d+)[.\s]*\n(?:\n+)'
    article_pattern = r'Статья\s+(\d+(?:\.\d+)?)'

    chapters = list(re.finditer(chapter_pattern, text))

    try:
        current_art_val = float(article_num)
    except ValueError:
        return "Неизвестно"

    for j, ch_match in enumerate(chapters):
        ch_start = ch_match.start()
        ch_end = chapters[j+1].start() if j+1 < len(chapters) else len(text)
        chapter_text = text[ch_start:ch_end]

        articles_in_chapter = re.findall(article_pattern, chapter_text)
        if articles_in_chapter:
            first_article = float(articles_in_chapter[0])
            last_article = float(articles_in_chapter[-1])

            if first_article <= current_art_val <= last_article:
                chapter_num = ch_match.group(1)
                chapter_name = ch_match.group(2).strip()
                return f"Глава {chapter_num}. {chapter_name}"

    return "Неизвестно"

if __name__ == "__main__":
    script_dir = pathlib.Path(__file__).parent.resolve()
    file_path = script_dir / "data" / "constitutionrf.docx"

    print(f"Поиск файла по пути: {file_path}")

    if not file_path.exists():
        print(f"Ошибка: Файл не найден")
    else:
        print("Загрузка документа...")
        text = load_constitution(str(file_path))

        print("Нарезка на чанки...")
        chunks = parse_articles(text)

    for chunk in chunks:
        chunk["chapter"] = find_chapter(text, chunk["article_num"])

    print(f"\nСтатей найдено: {len(chunks)}")

    output_json = script_dir / "data" / "chunks.json"

```

```

with open(output_json, "w", encoding="utf-8") as f:
    json.dump(chunks, f, ensure_ascii=False, indent=2)

print(f"\nЧанки сохранены в {output_json}")

```

Файл *vectorize.py*

```

import json
import pathlib
from sentence_transformers import SentenceTransformer
from qdrant_client import QdrantClient
from qdrant_client.models import Distance, VectorParams, PointStruct
import os
from dotenv import load_dotenv
load_dotenv()

def load_chunks(path: str) -> list[dict]:
    with open(path, "r", encoding="utf-8") as f:
        return json.load(f)

if __name__ == "__main__":
    script_dir = pathlib.Path(__file__).parent.resolve()

    chunks_path = script_dir / "data" / "chunks.json"
    print("Загрузка чанков...")
    chunks = load_chunks(str(chunks_path))
    print(f"Загружено чанков: {len(chunks)}")

    print("\nЗагрузка модели для эмбедингов...")
    model = SentenceTransformer("paraphrase-multilingual-mpnet-base-v2")
    print("Модель загружена")

    print("\nВекторизация чанков...")
    texts = [chunk["text"] for chunk in chunks]
    embeddings = model.encode(texts, show_progress_bar=True)
    print(f"Размерность эмбедингов: {embeddings.shape[1]}")

    client = QdrantClient(
        url=os.environ["QDRANT_URL"],
        api_key=os.environ["QDRANT_API_KEY"],
    )

    collection_name = "constitution"
    existing = [c.name for c in client.get_collections().collections]
    if collection_name in existing:
        print(f"Коллекция '{collection_name}' уже существует,
пересоздаём...")
        client.delete_collection(collection_name)

    client.create_collection(
        collection_name=collection_name,
        vectors_config=VectorParams(
            size=embeddings.shape[1],
            distance=Distance.COSINE

```

```

    )
)
print(f"Коллекция '{collection_name}' создана")

points = [
    PointStruct(
        id=i,
        vector=embeddings[i].tolist(),
        payload={
            "article_num": chunk["article_num"],
            "chapter": chunk["chapter"],
            "text": chunk["text"]
        }
    )
    for i, chunk in enumerate(chunks)
]

print(f"\nЗагрузка {len(points)} векторов в Qdrant...")
client.upsert(
    collection_name=collection_name,
    points=points
)

```

Файл *rag_pipeline.py*

```

import os
from dotenv import load_dotenv
from sentence_transformers import SentenceTransformer, CrossEncoder
from qdrant_client import QdrantClient
from ollama import Client as OllamaClient

load_dotenv()

embedder = SentenceTransformer("paraphrase-multilingual-mpnet-base-v2")
reranker = CrossEncoder("DiTy/cross-encoder-russian-msmarco")
qdrant = QdrantClient(
    url=os.environ["QDRANT_URL"],
    api_key=os.environ["QDRANT_API_KEY"],
)
ollama = OllamaClient()

def retrieve(query: str, top_k: int = 10) -> list[dict]:
    query_vector = embedder.encode(query).tolist()

    results = qdrant.query_points(
        collection_name="constitution",
        query=query_vector,
        limit=top_k
    ).points

    return [
        {
            "text": r.payload["text"],
            "article_num": r.payload["article_num"],

```

```

        "chapter": r.payload["chapter"],
        "score": r.score
    }
    for r in results
]

def rerank(query: str, candidates: list[dict], top_n: int = 3) -> list[dict]:
    pairs = [[query, c["text"]] for c in candidates]
    scores = reranker.predict(pairs)

    for i, doc in enumerate(candidates):
        doc["rerank_score"] = float(scores[i])

    reranked = sorted(candidates, key=lambda x: x["rerank_score"],
reverse=True)
    return reranked[:top_n]

def build_prompt(query: str, docs: list[dict]) -> str:
    context = ""
    for doc in docs:
        context += f"[Статья {doc['article_num']} | {doc['chapter']}] \n"
        context += f"{doc['text']} \n \n"

    prompt = f""""Ты – юридический ассистент, который помогает пользователям
находить информацию в Конституции Российской Федерации.

Тебе предоставлены релевантные статьи из Конституции РФ. Твоя задача – дать
точный и полезный ответ на вопрос пользователя, опираясь исключительно на
предоставленный контекст.

ОБЯЗАТЕЛЬНЫЕ ПРАВИЛА:
- Отвечай ТОЛЬКО на русском языке, независимо ни от каких обстоятельств
- Отвечай только на основе предоставленных статей, не придумывай информацию
- Если в контексте нет ответа на вопрос – честно скажи об этом
- Указывай номера статей, на которые опираешься
- Думай пошагово перед тем как дать финальный ответ

Контекст из Конституции РФ:
{context}
Вопрос пользователя: {query}

Ответ на русском языке: """"
    return prompt

def ask(query: str) -> dict:
    candidates = retrieve(query, top_k=10)

    top_docs = rerank(query, candidates, top_n=3)

    prompt = build_prompt(query, top_docs)
    response = ollama.chat(
        model="qwen2.5:7b",
        messages=[{"role": "user", "content": prompt}]

```

```

    )

    return {
        "answer": response.message.content,
        "sources": [
            {
                "article_num": d["article_num"],
                "chapter": d["chapter"],
                "rerank_score": round(d["rerank_score"], 4)
            }
            for d in top_docs
        ]
    }

if __name__ == "__main__":
    test_queries = [
        "Какие права имеет человек на свободу слова?",
        "Кто является верховным главнокомандующим?",
        "Сколько лет должно быть президенту?"
    ]

    for query in test_queries:
        print(f"\n{' '*60}")
        print(f"Вопрос: {query}")
        print(f"{' '*60}")
        result = ask(query)
        print(f"Ответ:\n{result['answer']}")
        print(f"\nИсточники:")
        for s in result["sources"]:
            print(f"    - Статья {s['article_num']} ({s['chapter']}) | rerank: {s['rerank_score']}")

```

Файл *api.py*

```

import os
from contextlib import asynccontextmanager
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from dotenv import load_dotenv
from sentence_transformers import SentenceTransformer, CrossEncoder
from qdrant_client import QdrantClient
from ollama import Client as OllamaClient
import re

load_dotenv()

embedder = None
reranker = None
qdrant = None
ollama = None

@asynccontextmanager

```

```

async def lifespan(app: FastAPI):
    global embedder, reranker, qdrant, ollama
    print("Загрузка моделей...")
    embedder = SentenceTransformer("paraphrase-multilingual-mpnet-base-v2")
    reranker = CrossEncoder("DiTy/cross-encoder-russian-msmarco")
    qdrant = QdrantClient(
        url=os.environ["QDRANT_URL"],
        api_key=os.environ["QDRANT_API_KEY"],
    )
    ollama = OllamaClient()
    print("Модели загружены, сервер готов")
    yield
    print("Сервер останавливается...")

app = FastAPI(
    title="Constitution RAG API",
    description="API для поиска информации в Конституции РФ",
    version="1.0.0",
    lifespan=lifespan
)

class QueryRequest(BaseModel):
    query: str

class Source(BaseModel):
    article_num: str
    chapter: str
    rerank_score: float

class QueryResponse(BaseModel):
    answer: str
    sources: list[Source]

def retrieve(query: str, top_k: int = 15) -> list[dict]:
    query_vector = embedder.encode(query).tolist()
    results = qdrant.query_points(
        collection_name="constitution",
        query=query_vector,
        limit=top_k
    ).points
    return [
        {
            "text": r.payload["text"],
            "article_num": r.payload["article_num"],
            "chapter": r.payload["chapter"],
            "score": r.score
        }
        for r in results
    ]

def rerank(query: str, candidates: list[dict], top_n: int = 5) -> list[dict]:
    pairs = [[query, c["text"]] for c in candidates]
    scores = reranker.predict(pairs)
    for i, doc in enumerate(candidates):

```



```

        doc["rerank_score"] = float(scores[i])
        reranked = sorted(candidates, key=lambda x: x["rerank_score"],
reverse=True)
        return reranked[:top_n]

def is_relevant(top_docs: list[dict], threshold: float = 0.05) -> bool:
    """Если лучший rerank-score ниже порога – вопрос не по Конституции"""
    if not top_docs:
        return False
    return top_docs[0]["rerank_score"] >= threshold

def build_prompt(query: str, docs: list[dict]) -> str:
    context = ""
    for doc in docs:
        context += f"[Статья {doc['article_num']} | {doc['chapter']}]\\n"
        context += f"{doc['text']}\\n\\n"

    return f""""Тебе предоставлены релевантные статьи из Конституции РФ. Твоя
задача – дать точный и полезный ответ на вопрос пользователя, опираясь
исключительно на предоставленный контекст.

```

ОБЯЗАТЕЛЬНЫЕ ПРАВИЛА:

- Отвечай ТОЛЬКО на русском языке
- Отвечай только на основе предоставленных статей, не придумывай информацию
- Если в контексте нет ответа – честно скажи об этом
- Указывай номера статей, на которые опираешься
- Думай пошагово перед финальным ответом

Контекст из Конституции РФ:

{context}

Вопрос пользователя: {query}

Ответ на русском языке: ""

```

OFF_TOPIC_PHRASES = [
    "что ты написал", "прошное сообщение", "системный промпт",
    "твой промпт", "предыдущее сообщение"
]

```

```

def clean_response(text: str) -> str:
    """Убираем только китайские символы"""
    text = re.sub(r'[\u4e00-\u9fff\u3400-\u4dbf\u00ff-\uffef]+' , '', text)
    return text.strip()

```

```
@app.get("/")
```

```
def root():
```

```
    return {"message": "Constitution RAG API работает"}
```

```
@app.post("/ask", response_model=QueryResponse)
```

```
def ask(request: QueryRequest):
```

```
    if not request.query.strip():
```

```
        raise HTTPException(status_code=400, detail="Запрос не может быть
пустым")

```

```
    query_lower = request.query.lower()
```

```

    if any(phrase in query_lower for phrase in OFF_TOPIC_PHRASES):
        return QueryResponse(
            answer="Я могу отвечать только на вопросы по тексту Конституции
Российской Федерации.",
            sources=[]
        )

    candidates = retrieve(request.query)
    top_docs = rerank(request.query, candidates)

    if not is_relevant(top_docs, threshold=0.01):
        return QueryResponse(
            answer="Ваш вопрос не относится к Конституции Российской
Федерации. Пожалуйста, задайте вопрос по тексту Конституции.",
            sources=[]
        )
    prompt = build_prompt(request.query, top_docs)

    response = ollama.chat(
        model="qwen2.5:7b",
        messages=[
            {
                "role": "system",
                "content": "Ты — юридический ассистент по Конституции РФ.
Отвечай ТОЛЬКО на русском языке. Если вопрос не связан с Конституцией РФ —
вежливо откажись отвечать."
            },
            {
                "role": "user",
                "content": prompt
            }
        ]
    )

    raw_answer = response.message.content
    print("="*60)
    print("RAW ОТВЕТ МОДЕЛИ:")
    print(raw_answer)
    print("="*60)

    cleaned_answer = clean_response(raw_answer)
    print("ПОСЛЕ ОЧИСТКИ:")
    print(cleaned_answer)
    print("="*60)

    return QueryResponse(
        answer=clean_response(response.message.content),
        sources=[
            Source(
                article_num=d["article_num"],
                chapter=d["chapter"],
                rerank_score=round(d["rerank_score"], 4)
            )
            for d in top_docs
        ]
    )

```

)

Файл *app.py*

```
import streamlit as st
import requests

API_URL = "http://localhost:8000/ask"

st.set_page_config(
    page_title="Конституция РФ – База знаний",
    page_icon="🏛️",
    layout="centered"
)

st.title("🏛️ База знаний: Конституция РФ")
st.caption("Задайте вопрос – система найдёт ответ в тексте Конституции Российской Федерации")

# История чата в session_state
if "messages" not in st.session_state:
    st.session_state.messages = []

# Отображаем историю
for msg in st.session_state.messages:
    with st.chat_message(msg["role"]):
        st.markdown(msg["content"])
        if "sources" in msg:
            with st.expander("📖 Источники"):
                for s in msg["sources"]:
                    st.markdown(f"**Статья {s['article_num']}** – {s['chapter']}")

# Поле ввода
if query := st.chat_input("Введите вопрос по Конституции РФ..."):

    # Показываем вопрос пользователя
    with st.chat_message("user"):
        st.markdown(query)
    st.session_state.messages.append({"role": "user", "content": query})

    # Запрашиваем API
    with st.chat_message("assistant"):
        with st.spinner("Ищу ответ..."):
            try:
                response = requests.post(API_URL, json={"query": query})
                response.raise_for_status()
                data = response.json()

                answer = data["answer"]
                sources = data["sources"]

                st.markdown(answer)
                with st.expander("📖 Источники"):
                    for s in sources:
```

```

        st.markdown(f"**Статья {s['article_num']}" –
{s['chapter']})

        st.session_state.messages.append({
            "role": "assistant",
            "content": answer,
            "sources": sources
        })

    except requests.exceptions.ConnectionError:
        st.error("❌ Не удалось подключиться к API. Убедитесь что
сервер запущен.")
    except Exception as e:
        st.error(f"❌ Ошибка: {e}")

```